

SketchFlow - overview

The Vision

SketchFlow is a real-time visual collaboration platform that seamlessly bridges brainstorming, communication, and task management into one unified workspace. Teams brainstorm on an infinite whiteboard, chat in real-time, and convert sticky notes into actionable tasks—all without switching between apps.

Final Technology Stack

Layer	Technology	Hosting	Purpose
Frontend	React + React-Konva	Vercel	UI rendering, canvas manipulation
Real-Time	Socket.IO	Render	Real-time sync, presence, events
Backend API	Express.js	Render	REST endpoints, business logic
Database	PostgreSQL	Neon/Supabase	All persistent data
Cache	Redis	Render/Railway	Sessions, locks, presence, event log
File Storage	Cloudinary	Cloudinary	Uploads, CDN, optimization
Email	Nodemailer	Render	Synchronous email notifications
Scheduling	node-cron	Render	Cleanup tasks
Containerization	Docker	Optional	Development consistency

What We're Using & Why

Component	Technology	Rationale
Whiteboard Shapes	React-Konva + Socket.IO events	Simple, battle-tested, immediate updates

Component	Technology	Rationale
Shape Sync	Versioned events (optimistic locking)	Last-write-wins is fine for visual elements
Sticky Note Text	Locking mechanism (no Yjs)	Simultaneous editing is rare; locking provides clear UX
Element Locks	Redis with TTL + heartbeats	Auto-expiry prevents stale locks
Conflict Resolution	Version numbers	More reliable than timestamps
Presence	Socket.IO + Redis	Lightweight, TTL-based cleanup
Chat	Socket.IO	Append-only, no conflicts
Tasks	REST + Socket.IO notifications	Need ACID, referential integrity
Task Updates	Optimistic locking (version column)	Prevent concurrent update conflicts
Event Log	Redis (60s TTL)	Replay missed events on reconnection
Rate Limiting	Socket.IO + Express middleware	Prevent abuse, protect server
File Uploads	Cloudinary (direct upload)	No server processing, CDN delivery
Reconnection	Exponential backoff + jitter	Prevent server stampede
Emails	Nodemailer (synchronous)	Simple, sufficient for MVP
Scheduled Tasks	node-cron	Direct database cleanup
Hosting	Vercel + Render + Neon	Zero-cost, production-ready

What We're NOT Using & Why

Technology	Reason
Yjs	Overkill for brainstorming notes; locking solves text conflicts simpler

Technology	Reason
BullMQ	Not needed for MVP; can be added later for exports/background jobs
WebRTC	More complex; <u>Socket.IO</u> is sufficient
AWS S3	Cloudinary is simpler for personal projects
MongoDB/Cassandra	Need relational data (tasks, users, workspaces)
GraphQL	REST is simpler for this scope
Kafka/RabbitMQ	BullMQ would be added before these
CRDTs	Versioned events + locking handles conflicts

Key Data Flow Patterns

1. Optimistic Update (Shapes)

text

```
User drags shape → UI updates immediately → Socket.IO emits ELEMENT_MOVED
→ Server validates version → Updates PostgreSQL → Broadcasts to room
→ Other users see shape move
```

2. Lock + Release (Sticky Notes)

text

```
User clicks sticky → Request lock → Redis SETNX with 30s TTL
→ Lock granted → User edits → Heartbeat every 10s
→ User blurs → Unlock → Save text via STICKY_TEXT_UPDATED
→ Others see "Alex is editing" and cannot edit
```

3. REST + Socket.IO (Tasks)

text

```
User converts sticky to task → REST API POST /tasks/convert
→ Create task in PostgreSQL → Emit TASK_CREATED via Socket.IO
```

→ All users see new task instantly

4. Reconnection with Event Replay

text

User reconnects → Socket.IO reconnects with jitter
→ Request missed events since last timestamp
→ Server replays from Redis event log (60s TTL)
→ Board state catches up

5. File Upload (Cloudinary)

text

User selects file → Cloudinary Upload Widget opens
→ Uploads directly to Cloudinary → Returns public_id/URL
→ Frontend sends public_id to Express API
→ Server stores metadata in PostgreSQL
→ Server emits FILE_UPLOADED via Socket.IO
→ All users see file in workspace

Architecture Classification

Primary Type: Hybrid Event-Driven Real-Time Modular Monolith

What This Means:

- **Event-Driven:** Socket.IO events drive real-time updates
- **Real-Time:** WebSocket connections for instant sync
- **Modular Monolith:** Single codebase with clear module boundaries
- **Synchronous:** Email sending happens in the request cycle
- **Scheduled:** Cron jobs handle periodic cleanup tasks

Database Schema (Core Tables)

sql

```

-- Core
users (id, email, name, password_hash, created_at)
workspaces (id, name, created_by, created_at)
workspace_members (workspace_id, user_id, role, joined_at)
boards (id, name, workspace_id, created_at)

-- Whiteboard (with versioning)
elements (
  id, board_id, type, x, y, width, height,
  color, text, version, created_at, updated_at, deleted_at
)

-- Tasks (with optimistic locking)
tasks (
  id, title, description, status, assignee_id,
  due_date, board_id, created_by, version,
  created_at, updated_at, deleted_at
)
task_sources (task_id, element_id, snapshot_text, created_at)

-- Collaboration
chat_messages (id, board_id, user_id, message, parent_id, created_at)
files (id, board_id, name, public_id, uploaded_by, created_at)
activity_logs (id, board_id, user_id, action, metadata, created_at)

```

Redis Key Structure

redis

```

# Presence
presence:board:{boardId} -> { userId, name } [TTL: 60s]

# Cursors
cursor:board:{boardId}:{userId} -> { x, y } [TTL: 5s]

# Locks (auto-expiring)
lock:element:{elementId} -> { userId } [TTL: 30s]

# Event Log (for replay)
board:events:{boardId} -> [event1, event2] [TTL: 60s]

# Board State Cache
board:state:{boardId} -> { elements: [...] } [TTL: 5s]

# Rate Limiting
rate:board:{boardId}:{userId} -> 15 [TTL: 1s]

```

```
# Sessions
session:{sessionId} -> { userId }
```

Hosting Strategy (Free Tier)

Component	Platform	Monthly Cost
Frontend	Vercel	\$0
Backend	Render	\$0
Database	Neon/Supabase	\$0
Redis Cache	Render/Railway	\$0
File Storage	Cloudinary	\$0
Total	-	\$0

Development Phases

Phase	Duration	Focus
1	Week 1	Auth, Workspaces, Boards, Basic REST APIs
2	Week 2	Whiteboard (Konva + Socket.IO + versioning)
3	Week 3	Sticky notes (locking + text) + Presence
4	Week 4	Tasks (conversion, CRUD, real-time)
5	Week 5	Chat + File uploads (Cloudinary) + Activity logging
6	Week 6	RBAC + Cron jobs + Polish + Deployment

Production Hardening Checklist

Item	Status	Implementation
Event Log + Replay	✓	Redis with 60s TTL
Rate Limiting	✓	Socket.IO + Express middleware
Graceful Shutdown	✓	SIGTERM handling with cleanup

Item	Status	Implementation
Optimistic Locking	✓	Version column in elements and tasks
Lock Heartbeat	✓	Check lock exists before renewing
Connection Pool	✓	Proper pool configuration
Environment Validation	✓	Validate all env vars on startup
Reconnection Jitter	✓	Exponential backoff + jitter
Memory Monitoring	✓	Monitor heap usage

Key Differentiators

Feature	Why It Matters
Sticky Note → Task Conversion	Ideas don't get lost; they evolve into trackable work
Optimistic Updates	App feels instant and responsive
Version-Based Conflict Resolution	Handles race conditions without complex CRDTs
Element Locking	Clear UX for collaborative editing
Event Log + Replay	Reliable real-time sync even after disconnection
Direct Cloudinary Uploads	No server bottleneck; optimized delivery
Zero-Cost Hosting	Fully functional production app for free